

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

INGENIERÍA MECATRÓNICA



**LABORATORIO : INTELIGENCIA ARTIFICIAL Y VISIÓN POR
COMPUTADORA**

1MTR56

AUTOMATIZACIÓN INDUSTRIAL INTELIGENTE B

Tabla de contenido

1	Inteligencia Artificial: Redes Neuronales Convolucionales	2
1	Introducción	2
2	Visión por computadora	3
2.1	Definición	3
2.2	Enfoque clásico mediante extracción manual de características	3
2.3	Enfoque moderno basado en aprendizaje profundo	4
3	Redes Neuronales Convolucionales	4
3.1	Capas convolucionales	6
3.2	Stride y padding	6
3.3	Capas de pooling	6
3.4	Funciones de activación	7
3.5	Capas totalmente conectadas	7
4	Proceso de entrenamiento	8
4.1	Inicialización	8
4.2	Forward propagation	8
4.3	Función de pérdida	8
4.4	Backpropagation	9
4.5	Optimización	9
5	Evaluación del modelo	9
5.1	Conjunto de validación	9
5.2	Métricas de rendimiento	9
5.3	Interpretación de métricas con conjuntos de datos pequeños	10
6	Preprocesamiento y aumentación de datos	11
6.1	Normalización de la entrada	11
6.2	Data augmentation	11
6.3	Batch Normalization	12
6.4	Regularización mediante Dropout	12
7	Experiencia guiada	12
7.1	Notebook 1 — notebook1_cnn_desde_cero.ipynb	13
7.2	Notebook 2 — notebook2_data_augmentation.ipynb	13
7.3	Construcción del dataset propio: tapitas	14
7.4	Notebook 3 — notebook3_clasificador_tapitas.ipynb	15
7.5	Cierre: discusión crítica	15
8	Cierre: el límite del entrenamiento desde cero	16
9	Anexos	17
2	Visión por Computadora: Transfer Learning y Beckhoff	21
10	Introducción	21
11	Industria 5.0 y robots colaborativos	22

11.1 Del paradigma 4.0 al 5.0	22
11.2 Robots colaborativos (cobots)	22
11.3 Normativa de seguridad: ISO/TS 15066	22
12 Reutilización del dataset de tapitas	23
13 Aprendizaje por transferencia	24
13.1 Motivación	24
13.2 MobileNetV2 como backbone	25
13.3 Feature extraction y fine-tuning	25
14 Entrenamiento con Transfer Learning	26
14.1 Generadores con aumentación de datos	26
14.2 Fase 1: Feature extraction	26
14.3 Fase 2: Fine-tuning	27
15 Protocolo ADS y comunicación con Beckhoff	28
15.1 El protocolo ADS	28
15.2 Configuración previa en TwinCAT	29
15.3 La librería pyads	29
16 Integración: del clasificador a la acción del brazo	30
16.1 Contrato de comunicación clasificador ↔ PLC	30
16.2 Bucle principal de inferencia y comunicación	30
17 Experiencia guiada	32
18 Anexos	33

Parte 1

Inteligencia Artificial: Redes Neuronales Convolucionales

1 Introducción

En la presente guía se introduce al estudiante en los fundamentos de la visión por computadora basada en aprendizaje profundo, con énfasis en las *Redes Neuronales Convolucionales* (CNN), y se desarrolla una primera implementación práctica en Python sobre un problema de clasificación de imágenes industriales.

Esta guía constituye la **primera parte** del laboratorio combinado de Inteligencia Artificial y Visión por Computadora, y aporta **10 puntos** a la nota total del laboratorio (sobre 20). La segunda parte (Guía 2) introducirá el aprendizaje por transferencia con MobileNetV2 para la clasificación de tapitas de colores e integrará el sistema con un controlador industrial Beckhoff vía el protocolo ADS.

Objetivos del laboratorio:

- Comprender los fundamentos teóricos de la visión por computadora moderna basada en aprendizaje profundo.
- Conocer la arquitectura y los componentes de una Red Neuronal Convolutiva.
- Asimilar el proceso de entrenamiento de una red neuronal: *forward propagation*, función de pérdida, *backpropagation* y optimización.
- Implementar en TensorFlow/Keras una CNN simple para un problema de clasificación de imágenes.
- Evaluar el rendimiento de un modelo entrenado mediante curvas de aprendizaje y matriz de confusión.

Conocimientos previos recomendados:

- Programación en Python (manejo de paquetes y entornos virtuales).
- Álgebra lineal básica (vectores, matrices, producto interior).
- Cálculo diferencial (gradiente, regla de la cadena).
- Nociones de procesamiento de señales discretas (operación de convolución 1D y 2D).

Materiales:

- Computadora con Python 3.10 o superior y entorno virtual del laboratorio activo.
- Dataset de botellas (vidrio y plástico), provisto en `data/botellas/`.
- Dataset de tapitas de colores, capturado por la propia pareja (Notebook 3).

- Notebooks de trabajo `parte1_ia/notebook1_cnn_desde_cero.ipynb`, `parte1_ia/notebook2` y `parte1_ia/notebook3_clasificador_tapitas.ipynb`.
- Librerías `tensorflow`, `numpy`, `matplotlib`, `seaborn`.

Visión general del laboratorio combinado. La Figura 1 resume el flujo completo del sistema que se construirá entre la Guía 1 y la Guía 2. La presente guía cubre los fundamentos teóricos y la implementación de la CNN base; la Guía 2 introducirá la clasificación con MobileNetV2 y la integración con el controlador industrial.

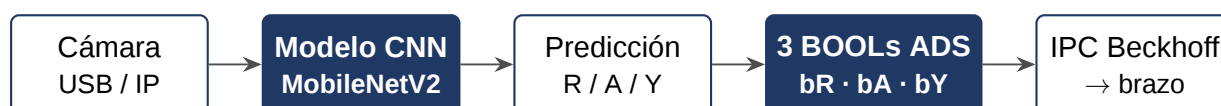


Figura 1. Pipeline end-to-end del laboratorio combinado IA + Visión por Computadora. La cámara captura la imagen del objeto a clasificar; la CNN entrenada por la pareja predice la clase; el resultado se escribe en tres variables booleanas del IPC Beckhoff vía ADS, las cuales activan la salida digital que dispara el ciclo del brazo robótico.

2 Visión por computadora

2.1 Definición

La visión por computadora es una disciplina dentro de la inteligencia artificial cuyo objetivo es dotar a las máquinas de la capacidad de interpretar el mundo visual de forma análoga al sistema visual humano. Mediante la combinación de cámaras, algoritmos y poder de cómputo, esta disciplina busca extraer información significativa a partir de imágenes y secuencias de video.

Las aplicaciones industriales son numerosas: control de calidad por inspección visual, identificación y trazabilidad de productos, asistencia a operarios mediante realidad aumentada, supervisión de procesos críticos, y guiado de robots colaborativos. En todos estos casos, la imagen actúa como una señal sensorial cuya interpretación habilita la toma de decisiones automatizada.

2.2 Enfoque clásico mediante extracción manual de características

El enfoque tradicional de la visión por computadora se basa en el diseño manual de descriptores que capturan información geométrica o estadística de la imagen (Figura 2). Entre las técnicas más representativas se encuentran la segmentación por color en el espacio HSV, la transformada de Hough para la detección de líneas y círculos, y descriptores como SIFT (*Scale-Invariant Feature Transform*) y SURF (*Speeded Up Robust Features*) para la detección de puntos de interés invariantes a escala y rotación.

Estos métodos resultan eficaces y computacionalmente eficientes cuando las condiciones del entorno son controladas: iluminación estable, fondo conocido y geometría predecible de los objetos. Sin embargo, su principal limitación es la fragilidad ante variaciones no anticipadas. Cambios de iluminación, oclusiones parciales, deformaciones, o la

introducción de nuevas clases de objetos requieren típicamente un rediseño manual del pipeline.

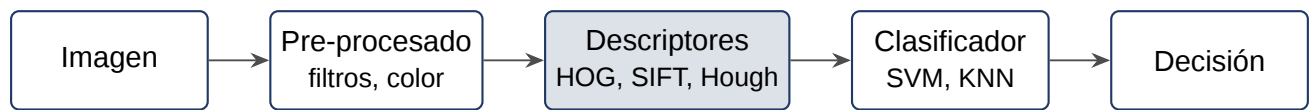


Figura 2. Pipeline clásico de visión por computadora: las características son diseñadas manualmente por el ingeniero.

2.3 Enfoque moderno basado en aprendizaje profundo

El surgimiento del aprendizaje profundo, y en particular de las Redes Neuronales Convolucionales, modificó radicalmente este paradigma. En lugar de diseñar manualmente los descriptores, una CNN aprende **directamente de los datos** qué características son relevantes para la tarea (Figura 3). Las primeras capas tienden a aprender detectores de bordes y texturas, mientras que las capas más profundas aprenden representaciones cada vez más abstractas (partes, objetos enteros).

Este enfoque presenta tres ventajas frente al método tradicional. Primero, permite manejar variaciones complejas (iluminación, ángulos, escalas) sin reformular el algoritmo. Segundo, escala de manera natural a problemas multiclase. Tercero, aprovecha la disponibilidad creciente de grandes conjuntos de datos y de cómputo paralelo en GPU, lo que ha permitido alcanzar y superar el desempeño humano en numerosas tareas de clasificación visual.



Figura 3. Pipeline moderno basado en aprendizaje profundo: la red aprende las características óptimas para la tarea directamente de los datos.

Analogía mecatrónica. En control clásico, el diseñador define manualmente la función de transferencia del controlador. En control basado en aprendizaje, los parámetros del controlador se ajustan a partir de datos. Las CNN ocupan el papel de un controlador adaptativo sobre el dominio visual.

3 Redes Neuronales Convolucionales

Las CNN son una clase de redes neuronales especializadas en el procesamiento de datos con estructura de rejilla, como imágenes (rejilla 2D de píxeles con canales de color). Se diferencian de las redes neuronales totalmente conectadas tradicionales por la introducción de **capas convolucionales**, que aprovechan la localidad espacial y la invariancia traslacional inherentes a las imágenes.

La Figura 4 ilustra la estructura genérica de una CNN: la imagen de entrada atraviesa una secuencia alternada de capas convolucionales y de *subsampling* (pooling) que

producen mapas de características (*feature maps*) progresivamente más abstractos, hasta llegar a una capa totalmente conectada que produce la salida final.

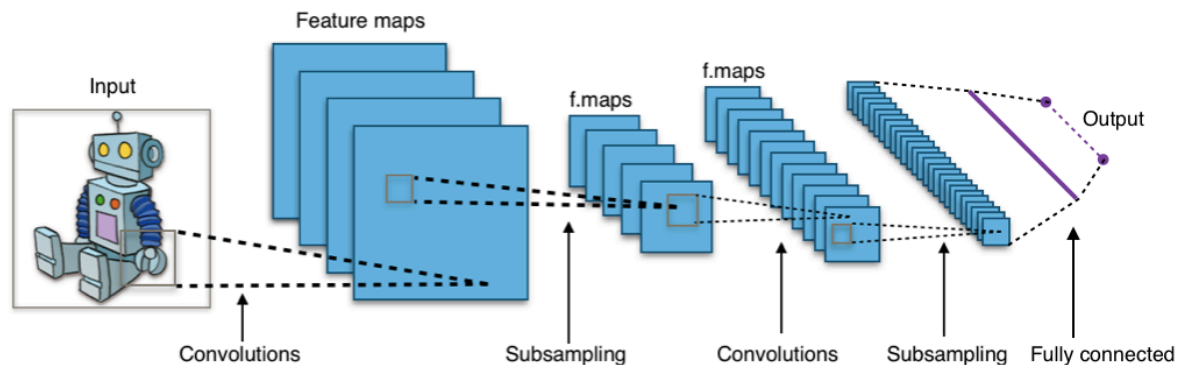


Figura 4. Estructura genérica de una red neuronal convolucional. La entrada (imagen) se procesa mediante etapas sucesivas de convolución y *subsampling*, generando representaciones internas (*feature maps*) cada vez más abstractas, hasta la capa totalmente conectada que produce la salida. Fuente: Aphex34, Wikimedia Commons (CC BY-SA 4.0).

La Figura 5 presenta la arquitectura específica que se implementará en la experiencia guiada (Sección 7). Es una CNN **sencilla, entrenada desde cero**: dos etapas de convolución y *max pooling* que extraen mapas de características cada vez más compactos, seguidas de un clasificador totalmente conectado. Partiendo de una imagen de entrada de $64 \times 64 \times 3$, cada capa Conv2D de 32 filtros 3×3 reduce el mapa a 62×62 y luego a 29×29 (convolución *valid*), y cada MaxPooling2D de 2×2 lo lleva a 31×31 y 14×14 respectivamente. El resultado se aplanan (Flat) y se conecta a una capa densa de 128 neuronas con activación *ReLU* y *Dropout* de 0.5, y finalmente a una capa de salida de 2 neuronas con activación *softmax* (vidrio / plástico).

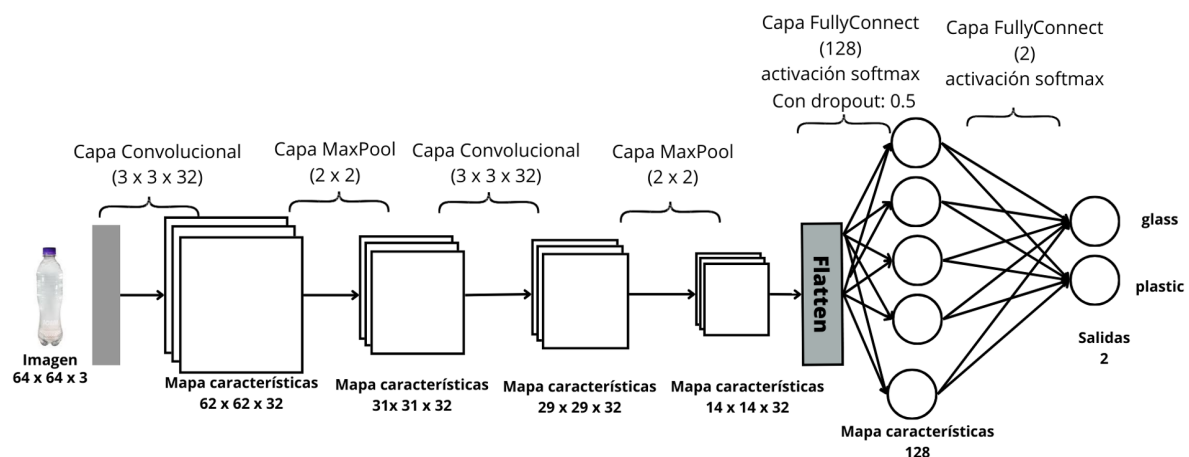


Figura 5. Arquitectura de la CNN entrenada desde cero en la experiencia guiada (clasificador de botellas). Dos bloques Conv2D(32, 3×3) + MaxPooling2D(2×2) extraen las características; tras Flatten, una capa densa de 128 neuronas (*ReLU*) con *dropout* 0.5 y una capa de salida *softmax* de 2 neuronas producen la clasificación. Fuente: elaboración propia.

3.1 Capas convolucionales

Una capa convolucional aplica un conjunto de **kernels** (también llamados filtros) sobre la imagen de entrada. Cada kernel es una pequeña matriz de pesos (típicamente 3×3 o 5×5) que se desliza espacialmente sobre la entrada realizando, en cada posición, una multiplicación elemento a elemento seguida de una suma. El resultado es un **mapa de características** (*feature map*) que resalta la presencia del patrón codificado en el kernel. La Figura 6 ilustra la operación con un kernel Sobel- x clásico para la detección de gradientes horizontales.

Durante el entrenamiento, los pesos de los kernels se ajustan automáticamente para que la red aprenda los patrones más útiles para la tarea. Una capa con N filtros produce N mapas de características, formando un tensor de salida de profundidad N .

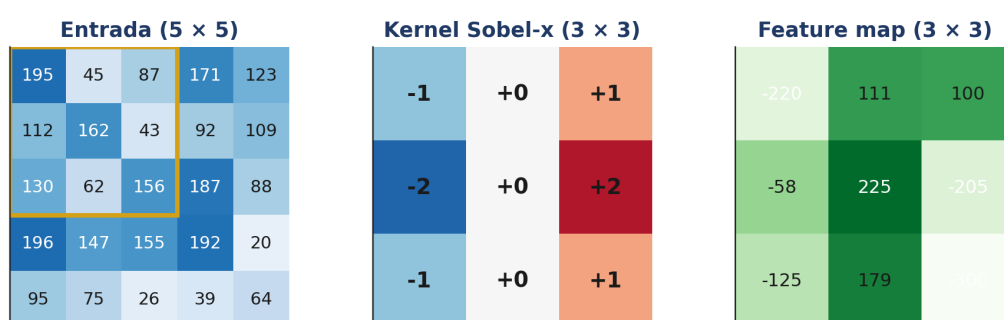


Figura 6. Operación de convolución 2D. El kernel Sobel- x aplicado a una región de la imagen produce un único valor en el *feature map*, repitiéndose la operación para cada posición espacial. Fuente: elaboración propia.

3.2 Stride y padding

El **stride** indica el paso con el que el kernel se desplaza sobre la imagen. Un stride de 1 produce una salida del mismo tamaño espacial que la entrada (asumiendo padding adecuado), mientras que un stride de 2 reduce las dimensiones espaciales a la mitad.

El **padding** consiste en añadir píxeles, típicamente con valor cero, en los bordes de la imagen antes de la convolución. El modo `same` ajusta el padding para preservar las dimensiones espaciales de salida; el modo `valid` no añade padding y reduce la salida en función del tamaño del kernel.

3.3 Capas de pooling

Las capas de pooling reducen las dimensiones espaciales de los mapas de características, conservando la información más relevante. **Max pooling** retiene el valor máximo dentro de cada ventana, mientras que **average pooling** retiene el promedio (Figura 7). El pooling cumple tres funciones: reducir el número de parámetros y el costo computacional de las capas posteriores, mitigar el sobreajuste, e introducir un grado de invariancia a pequeñas traslaciones.

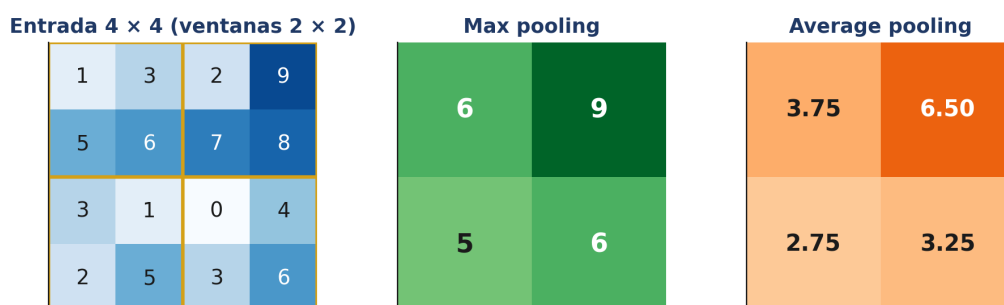


Figura 7. Comparativa entre max pooling y average pooling sobre una entrada 4×4 con ventanas 2×2 y stride 2. Fuente: elaboración propia.

3.4 Funciones de activación

Las funciones de activación introducen no-linealidades en la red, sin las cuales una pila de capas convolucionales colapsaría a una única transformación lineal y la red sería incapaz de aprender relaciones complejas. La función más utilizada es **ReLU** (*Rectified Linear Unit*), definida como $f(x) = \max(0, x)$, por su simplicidad computacional y por mitigar el problema del desvanecimiento del gradiente. Variantes como Leaky ReLU permiten pequeños valores negativos, atenuando el problema de las “neuronas muertas”. La Figura 8 muestra las cuatro funciones de activación más empleadas en redes convolucionales.

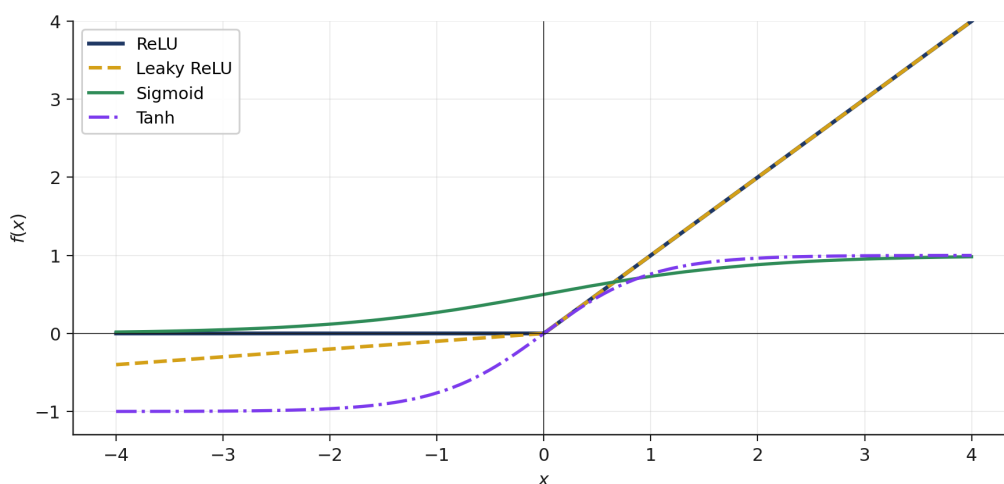


Figura 8. Funciones de activación más empleadas en redes convolucionales. Fuente: elaboración propia.

3.5 Capas totalmente conectadas

Al final de la red, las capas *fully connected* (también llamadas densas) combinan las características aprendidas por las capas convolucionales y producen la salida final. En problemas de clasificación, la última capa densa tiene tantas neuronas como clases, y emplea la función de activación **softmax**, que normaliza las salidas para que sumen 1, interpretándose como una distribución de probabilidad sobre las clases.

4 Proceso de entrenamiento

El entrenamiento de una CNN consiste en ajustar iterativamente los pesos de la red (los valores de los kernels y de las capas densas) para minimizar una función de pérdida que mide la discrepancia entre las predicciones de la red y las etiquetas reales. El proceso es inherentemente iterativo y se ilustra en la Figura 9.

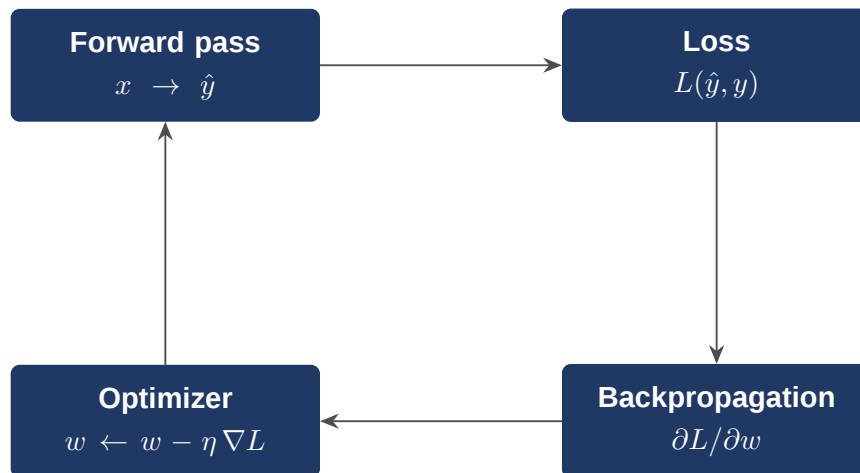


Figura 9. Ciclo de entrenamiento iterativo de una red neuronal. Cada iteración recorre las cuatro etapas hasta minimizar la función de pérdida.

4.1 Inicialización

Los pesos de la red se inicializan típicamente con valores aleatorios pequeños, siguiendo esquemas como *Glorot* (Xavier) o *He*, diseñados para que la varianza de las activaciones se mantenga estable a lo largo de las capas. Una inicialización adecuada es crítica para garantizar una convergencia eficiente.

4.2 Forward propagation

La imagen de entrada se propaga capa por capa a través de la red. Cada capa transforma su entrada según los pesos actuales y la función de activación correspondiente, produciendo finalmente la predicción $\hat{y}$.

4.3 Función de pérdida

La función de pérdida cuantifica el error de la predicción respecto a la etiqueta real. Para clasificación multiclase se utiliza típicamente la **entropía cruzada** (*categorical cross-entropy*), que mide la divergencia entre la distribución predicha y la distribución real (codificada como vector *one-hot*):

$$L(\hat{y}, y) = - \sum_{i=1}^C y_i \log \hat{y}_i$$

donde C es el número de clases. Para regresión se emplean el error cuadrático medio (MSE) o el error absoluto medio (MAE).

4.4 Backpropagation

El algoritmo de *backpropagation* calcula el gradiente de la función de pérdida respecto a cada uno de los pesos de la red, aplicando la regla de la cadena del cálculo diferencial. Estos gradientes indican la dirección y magnitud en que cada peso debe ajustarse para reducir el error.

4.5 Optimización

Los **optimizadores** son los algoritmos que actualizan los pesos a partir de los gradientes calculados. El más simple es el descenso de gradiente estocástico (**SGD**), que aplica la regla $w \leftarrow w - \eta \cdot \nabla L$, donde η es la tasa de aprendizaje (*learning rate*). Variantes adaptativas como **Adam** o **RMSprop** ajustan automáticamente la tasa de aprendizaje por parámetro, y son particularmente efectivas en redes profundas. Adam es, en la práctica, el optimizador por defecto en la mayoría de aplicaciones modernas.

Analogía mecatrónica. El optimizador puede interpretarse como un controlador que regula la trayectoria de los pesos hacia el mínimo de la función de pérdida. La tasa de aprendizaje cumple el papel de la ganancia proporcional: una ganancia excesiva genera inestabilidad, mientras que una ganancia insuficiente produce convergencia lenta.

Una lista de optimizadores y sus casos de uso se presenta en el **Anexo A1**.

5 Evaluación del modelo

5.1 Conjunto de validación

El dataset disponible se divide en al menos dos subconjuntos disjuntos: **entrenamiento** (típicamente 70–80 % de los datos) y **validación** (20–30 % restante). El modelo ajusta sus pesos únicamente sobre el conjunto de entrenamiento, y se evalúa periódicamente sobre el conjunto de validación para estimar su capacidad de **generalización**, es decir, su desempeño sobre datos no observados durante el entrenamiento.

La discrepancia entre el desempeño de entrenamiento y el de validación es un indicador clave: si el modelo obtiene alto desempeño en entrenamiento pero bajo en validación, se encuentra en régimen de **sobreajuste** (*overfitting*) y ha memorizado el dataset sin aprender los patrones generales (Figura 10).

5.2 Métricas de rendimiento

Para problemas de clasificación, las métricas más utilizadas son:

- **Accuracy:** proporción de predicciones correctas. Adecuada cuando las clases están balanceadas.
- **Precision:** de las predicciones positivas, qué fracción es realmente positiva.
- **Recall:** de los positivos reales, qué fracción fue correctamente detectada.
- **F1-score:** media armónica entre precision y recall, útil cuando existe desbalance

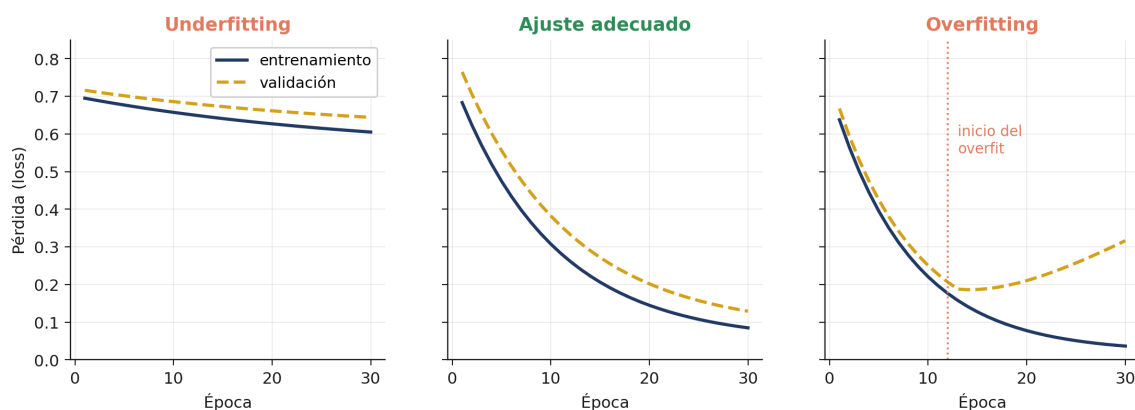


Figura 10. Diagnóstico de entrenamiento mediante curvas de pérdida. *Underfitting*: el modelo no logra reducir la pérdida ni siquiera en entrenamiento. *Ajuste adecuado*: ambas curvas descienden y convergen. *Overfitting*: la pérdida de entrenamiento sigue bajando pero la de validación comienza a aumentar. Fuente: elaboración propia.

de clases.

- **Matriz de confusión:** tabla cruzada de clases reales vs. predichas, que permite identificar confusiones específicas entre pares de clases.

Un detalle de métricas adicionales se presenta en el **Anexo A4**.

5.3 Interpretación de métricas con conjuntos de datos pequeños

En laboratorios con datasets reducidos —decenas o pocos cientos de imágenes por clase— la *accuracy* sobre el conjunto de validación debe interpretarse con cautela. Conviene tener presentes tres consideraciones metodológicas que diferencian un análisis riguroso de una lectura ingenua de los resultados.

La accuracy es una estimación con incertidumbre. Medida sobre n imágenes de validación, la accuracy es una proporción muestral cuya desviación estándar aproximada es $\sqrt{p(1-p)/n}$. Con $n = 25$ imágenes, una accuracy del 80% conlleva una incertidumbre cercana a ± 8 puntos porcentuales, y cada imagen individual desplaza el resultado en 4%. En consecuencia, una diferencia pequeña de accuracy entre dos modelos evaluados sobre un conjunto reducido *no* es concluyente: debe acompañarse de la matriz de confusión y de las métricas por clase antes de afirmar que un modelo supera a otro.

La brecha entre entrenamiento y validación es más informativa que el valor absoluto. Una accuracy de entrenamiento elevada junto a una accuracy de validación sensiblemente menor (brecha mayor a 10–15 puntos) es la firma del *overfitting*: el modelo memoriza el conjunto de entrenamiento en lugar de generalizar. Esta brecha —y no la accuracy aislada— es el diagnóstico que orienta las decisiones de mejora: capturar más datos, aplicar aumentación, reforzar la regularización o, como se aborda en la Guía 2, recurrir al aprendizaje por transferencia.

Reproducibilidad y variabilidad. Aunque en los notebooks se fijan las semillas de los generadores aleatorios (random, numpy y tensorflow), pueden persistir pequeñas variaciones entre ejecuciones debido al orden de los lotes y al no-determinismo de ciertas operaciones numéricas. Las conclusiones del informe deben sostenerse sobre tendencias robustas y reproducibles, no sobre el último decimal de una única corrida.

6 Preprocesamiento y aumentación de datos

6.1 Normalización de la entrada

Las imágenes capturadas por una cámara digital contienen valores de intensidad de píxel en el rango entero $[0, 255]$ por canal. Antes de ser presentadas a una CNN, estos valores se *normalizan* al rango real $[0, 1]$ dividiendo cada píxel por 255. Esta normalización estabiliza el entrenamiento al mantener las activaciones de la red dentro de un rango acotado y evitar gradientes excesivamente grandes durante la *backpropagation*.

Algunos modelos pre-entrenados (como MobileNetV2, que se empleará en la Guía 2) requieren una normalización distinta, típicamente al rango $[-1, 1]$ mediante la transformación $x \mapsto 2 \cdot (x/255) - 1$, o la sustracción de la media por canal del dataset original. Es responsabilidad del estudiante aplicar la normalización adecuada según el modelo utilizado.

6.2 Data augmentation

Cuando el dataset disponible es reducido (centenares o pocos millares de imágenes), la red tiende a memorizar las muestras de entrenamiento en lugar de aprender patrones generalizables. Una técnica estándar para mitigar este problema es la **aumentación de datos** (*data augmentation*): generar variaciones sintéticas de las imágenes originales aplicando transformaciones que preservan la clase (rotaciones, traslaciones, cambios de escala, reflexiones horizontales, variaciones de brillo y contraste). Cada época, la red ve versiones ligeramente distintas de cada imagen, lo que mejora su capacidad de generalización.

Las transformaciones más utilizadas son:

- **Reflexión horizontal** (`horizontal_flip=True`): especialmente útil cuando la clase es simétrica respecto al eje vertical.
- **Rotaciones** (`rotation_range`): pequeñas rotaciones aleatorias, típicamente $\pm 10^\circ$ a $\pm 20^\circ$.
- **Traslaciones** (`width_shift_range`, `height_shift_range`): desplazamientos en porcentaje del tamaño de la imagen.
- **Zoom** (`zoom_range`): variaciones de escala que simulan diferentes distancias del objeto a la cámara.
- **Brillo / contraste**: simulan variaciones de iluminación.

Precaución con la elección de transformaciones. No todas las transformaciones son válidas para cualquier problema. Por ejemplo, una reflexión vertical podría no ser apropiada si la orientación del objeto es semánticamente relevante (un 6 reflejado verticalmente se convierte en un 9). En el caso de clasificación de tapitas por color, deben evitarse transformaciones que alteren el matiz (*hue shifting*), pues romperían la clase.

6.3 Batch Normalization

Batch Normalization (Ioffe & Szegedy, 2015) es una técnica que normaliza las activaciones de una capa para que tengan media cero y varianza unitaria sobre el *batch* actual durante el entrenamiento. Esto produce tres efectos beneficiosos: estabiliza la distribución de las activaciones a lo largo del entrenamiento, permite usar tasas de aprendizaje más altas, y actúa como una forma suave de regularización. En la práctica, su uso acelera notablemente la convergencia y mejora la generalización.

En una CNN, la convención más común es insertar una capa `BatchNormalization()` inmediatamente después de cada capa `Conv2D` o `Dense`, antes de la siguiente operación (pooling o nueva convolución). La arquitectura sencilla de la experiencia guiada (Figura 5) no la incorpora —para mantener el modelo mínimo y enfocar la atención en el flujo básico—, pero es una técnica de uso extendido en redes más profundas, como el *backbone* MobileNetV2 que se empleará en la Guía 2.

6.4 Regularización mediante Dropout

La regularización es un conjunto de técnicas que reducen el sobreajuste penalizando la complejidad del modelo. La técnica más utilizada en CNN es **Dropout**: durante el entrenamiento, una fracción aleatoria de las neuronas de una capa se desactiva (su salida se fuerza a cero) en cada paso. Esto obliga a la red a no depender excesivamente de neuronas individuales y a aprender representaciones distribuidas más robustas. En inferencia, el Dropout se desactiva automáticamente y todas las neuronas se utilizan.

La tasa típica de Dropout en las capas totalmente conectadas es entre 0.3 y 0.5. En las capas convolucionales se utilizan tasas más pequeñas, entre 0.1 y 0.25.

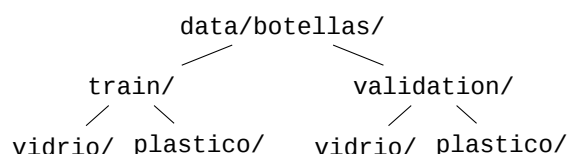
7 Experiencia guiada

En esta experiencia se construye una CNN para clasificar imágenes de botellas en dos categorías —**vidrio** y **plástico**— con el fin de asimilar el flujo completo de trabajo (preparación del dataset, definición del modelo, entrenamiento, evaluación y predicción) sobre un problema reducido, antes de abordar la Guía 2, donde se introduce el aprendizaje por transferencia (*transfer learning*) sobre un problema más complejo.

El trabajo práctico se desarrolla en **dos notebooks complementarios** que la pareja ejecuta y completa. La indicación general es la misma para ambos: **ejecutar el notebook de principio a fin, completar todas las celdas marcadas como «completar» y responder por escrito todas las preguntas conceptuales** que aparecen intercaladas. Cada notebook contiene la explicación detallada de cada celda; la presente sección

no reproduce ese contenido, sino que describe *qué* se realiza en cada etapa y *cuánto* aporta a la evaluación. El detalle de las entregas y la rúbrica se encuentran en la *Ficha de Evaluación* del laboratorio, que cubre ambas guías: la Guía 1 aporta **10 puntos** y la Guía 2 otros 10, para una nota total sobre 20.

La arquitectura es la CNN sencilla descrita en la Figura 5: dos bloques Conv2D(32, 3×3) + MaxPooling2D, seguidos de Flatten, una capa densa de 128 neuronas (*ReLU*) con *dropout* 0.5 y una capa de salida *softmax* de dos neuronas. El dataset se organiza en subcarpetas, una por clase, dentro de *train/* y *validation/*, de modo que *ImageDataGenerator* infiera las etiquetas automáticamente:



7.1 Notebook 1 — notebook1_cnn_desde_cero.ipynb

Recorre el ciclo completo de una CNN entrenada desde cero: carga y exploración del dataset, definición y compilación de la arquitectura, entrenamiento, evaluación mediante matriz de confusión y predicción sobre imágenes individuales. Las Figuras 11 y 12 ilustran, respectivamente, las curvas de aprendizaje y la matriz de confusión esperadas al concluir el entrenamiento.

Etapas del notebook	(secciones del notebook)	Qué se realiza	Puntos
1. Arquitectura y entrenamiento	(secciones 3–6)	Definir y compilar la CNN, entrenar y analizar las curvas de <i>loss/accuracy</i> y la matriz de confusión sobre validación.	1.5
2. Evaluación con datos propios	(sección 7)	Clasificar cuatro fotografías capturadas por la pareja en condiciones variadas e identificar aciertos y fallos del modelo.	1.5

7.2 Notebook 2 — notebook2_data_augmentation.ipynb

Retoma el clasificador anterior para evidenciar y mitigar el *overfitting*. El estudiante reproduce el sobreajuste del modelo base, experimenta con las transformaciones de aumentación y selecciona las pertinentes para el problema, reentrena con *data augmentation* y compara el desempeño frente al modelo sin aumentar.

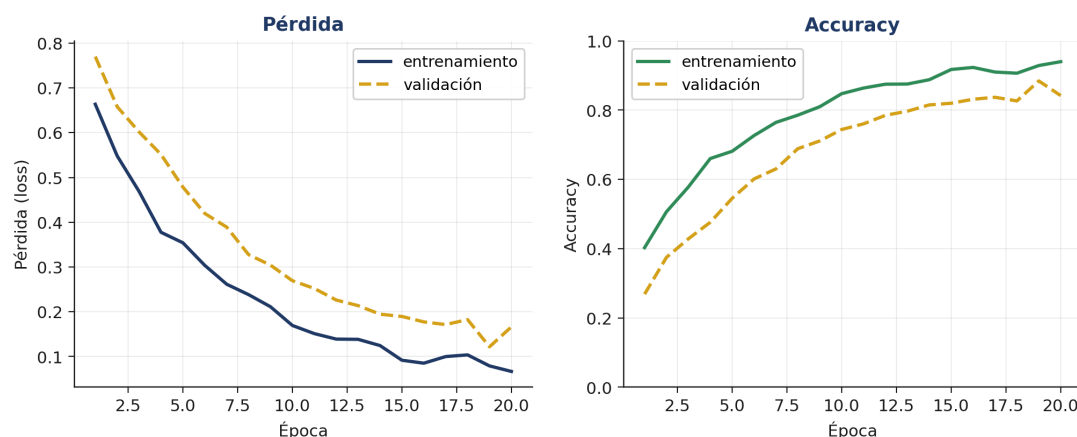


Figura 11. Curvas de aprendizaje esperadas tras el entrenamiento sobre el dataset de botellas. Fuente: elaboración propia.

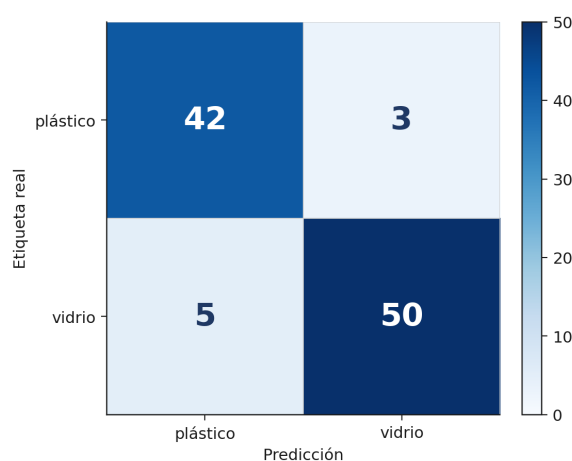


Figura 12. Matriz de confusión esperada sobre el conjunto de validación. La diagonal corresponde a predicciones correctas; los valores fuera de la diagonal, a confusiones entre clases. Fuente: elaboración propia.

Etapa (secciones notebook)	del Qué se realiza	Puntos
3. Data augmentation (secciones 2–5)	Diagnosticar el <i>overfitting</i> del modelo base, seleccionar transformaciones justificadas, reentrenar y comparar las curvas frente al modelo base.	2.5

7.3 Construcción del dataset propio: tapitas

A diferencia de los Notebooks 1 y 2, que emplean el dataset de botellas provisto, el **Notebook 3** introduce un dataset construido por la propia pareja: tapitas de colores (**rojo, azul, amarillo**) más una clase fondo. Este ejercicio reproduce las condiciones reales de un proyecto de control de calidad industrial: el equipo define el protocolo de

adquisición, captura las muestras y las organiza. El mismo dataset se reutilizará en la Guía 2 con aprendizaje por transferencia.

Protocolo de captura. Para garantizar la calidad del dataset:

- **Iluminación variada:** capturar en distintas condiciones de luz (natural, artificial cálida, fría).
- **Fondos múltiples:** variar el fondo (mesa del laboratorio, hoja blanca, hoja oscura).
- **Distancia y ángulo:** variar la distancia cámara–objeto (10–40 cm) y el ángulo.
- **Cantidad mínima:** al menos 50 muestras por clase, considerando que la aumentación multiplicará este número.
- **Una sola tapita por imagen:** no incluir varias tapitas (eso sería detección, no clasificación).

Extracción de frames. Una forma eficiente de obtener muchas muestras es grabar un video corto (15–30 s) por color y extraer los *frames*. El Notebook 3 realiza esta extracción de forma interactiva (grabación o subida del video → `data/tapitas/<clase>/`) y descarta automáticamente los frames borrosos. Las etiquetas se asignan en orden alfabético siguiendo la convención de `flow_from_directory`.

7.4 Notebook 3 — `notebook3_clasificador_tapitas.ipynb`

Aplica el flujo de los Notebooks 1 y 2 al dataset propio de tapitas: a partir de los videos extrae los frames, divide en *train/validation*, entrena una CNN desde cero (misma familia de arquitectura) y la evalúa. El modelo se exporta a `modelos/` para reutilizarlo en la Guía 2. El resultado —bueno en validación, pero limitado ante condiciones nuevas— es el punto de partida que motiva el aprendizaje por transferencia.

Etapa (pasos del notebook)	Qué se realiza	Puntos
4. Captura y preparación del dataset (pasos 1–2)	Grabar/subir un video por color, extraer los frames a <code>data/tapitas/</code> y reportar el conteo por clase.	1.0
5. Entrenamiento y evaluación CNN (pasos 3–4)	Dividir <i>train/validation</i> , entrenar la CNN desde cero, analizar curvas y matriz de confusión, y exportar el modelo.	2.0

7.5 Cierre: discusión crítica

A partir de los resultados de ambos notebooks, la pareja formula al menos dos hipótesis técnicas sobre las causas de los errores observados (sesgos del dataset, sensibilidad a la iluminación, limitaciones de la arquitectura) y propone al menos dos estrategias de mejora sin modificar la arquitectura propuesta. Esta discusión se documenta en la Ficha de Evaluación (**2.0 puntos**) y motiva el aprendizaje por transferencia que se

aborda en la Guía 2.

Distribución de la nota de la Guía 1	Puntos
Notebook 1 — notebook1_cnn_desde_cero.ipynb	3.0
Notebook 2 — notebook2_data_augmentation.ipynb	2.5
Notebook 3 — notebook3_clasificador_tapitas.ipynb	3.0
Discusión crítica	1.5
Total Guía 1	10.0

8 Cierre: el límite del entrenamiento desde cero

La evaluación sobre datos propios —tanto las fotografías de botellas (Notebook 1) como el clasificador de tapitas entrenado desde cero (Notebook 3)— suele revelar un resultado aleccionador: el modelo alcanza buena *accuracy* sobre validación, pero **falla al enfrentar condiciones reales** de iluminación, fondo o ángulo distintas a las del entrenamiento. La aumentación de datos (Notebook 2) mitiga parcialmente el problema, pero no lo resuelve: una CNN entrenada desde cero necesita, para generalizar bien, muchas más imágenes de las que es razonable capturar a mano.

Este es precisamente el límite que motiva la **Guía 2**. En lugar de aprender todos los filtros desde cero, se parte de una red ya entrenada sobre millones de imágenes y se reutiliza su conocimiento —el *aprendizaje por transferencia* (*transfer learning*)—, lo que permite obtener clasificadores robustos con apenas unos cientos de imágenes por clase. Sobre esa técnica se construye el sistema de control de calidad industrial de la segunda parte del laboratorio, reutilizando el dataset de tapitas del Notebook 3.

9 Anexos

Anexo A1. Optimizadores comunes

Optimizador	Descripción	Cuándo utilizarlo
SGD	Descenso de gradiente estocástico. Actualiza los pesos en la dirección opuesta al gradiente, escalado por el learning rate.	Modelos sencillos; control fino del LR; problemas donde Adam genera oscilaciones.
Adam	Combina momentum y RMSprop adaptativo.	Default razonable para la mayoría de problemas modernos, incluida visión por computadora.
RMSprop	Divide el gradiente por una media móvil de magnitudes recientes.	Redes recurrentes; gradientes con magnitud muy variable.
Adagrad	Aumenta el LR para parámetros poco frecuentes y lo reduce para los más comunes.	Datos dispersos o con muchas features categóricas.
Adadelta	Extensión de Adagrad que evita la disminución indefinida del LR.	Cuando Adagrad converge demasiado rápido a un LR pequeño.
Adamax	Variante de Adam basada en la norma infinito.	Redes profundas donde Adam es inestable.
Nadam	Adam con momento de Nesterov.	Problemas que se benefician del lookahead del momento de Nesterov.

Anexo A2. Funciones de pérdida

Función de pérdida	Descripción	Cuándo utilizarla
BinaryCrossentropy	Distancia entre la salida binaria predicha y la etiqueta real	Clasificación binaria con activación sigmoid
CategoricalCrossentropy	Distancia entre la distribución softmax y la etiqueta one-hot	Multiclase con etiquetas one-hot
SparseCategorical-Crossentropy	Equivalente a Categorical-Crossentropy pero acepta etiquetas enteras directamente	Multiclase sin necesidad de codificar one-hot
MeanSquaredError (MSE)	Promedio de los errores cuadrados	Regresión; penaliza fuertemente errores grandes
MeanAbsoluteError (MAE)	Promedio de los errores absolutos	Regresión robusta a outliers
Huber	Híbrida entre MSE y MAE	Regresión robusta con suavidad en el origen
Hinge	Pérdida con margen para SVM	Clasificación binaria con margen máximo
CosineSimilarity	Similitud por coseno entre vectores	Cuando importa la dirección y no la magnitud (embeddings)

Anexo A3. Parámetros principales del objeto model

Capa / argumento	Opciones	Sintaxis de ejemplo
Conv2D – filtros	32, 64, 128, 256, ...	Conv2D(32, ...)
Conv2D – kernel	(3,3), (5,5), (7,7)	Conv2D(32, (3, 3))
Conv2D – activación	relu, elu, selu, swish, tanh	Conv2D(32, (3, 3), activation='relu')
Conv2D input_shape	– (H, W, C) . Solo en la primera capa.	input_shape=(64, 64, 3)
MaxPooling2D pool_size	– (2,2), (3,3)	MaxPooling2D((2, 2))
MaxPooling2D strides	– (1,1), (2,2), ...	strides=(2, 2)
Flatten	Sin parámetros	Flatten()
Dense – units	32, 64, 128, 256, 512	Dense(128)
Dense – activación	relu, softmax, sigmoid, linear	Dense(128, activation='relu')
Dropout – rate	0.1 a 0.5	Dropout(0.5)
Capa de salida (clasificación)	N neuronas con softmax	Dense(N, activation='softmax')
compile – optimizer	ver Anexo A1	optimizer='adam'
compile – loss	ver Anexo A2	loss='categorical_crossentropy'
compile – metrics	ver Anexo A4	metrics=['accuracy']

Anexo A4. Métricas de evaluación

Métrica	Descripción	Cuándo utilizarla
Accuracy	Proporción de predicciones correctas	Clasificación con clases balanceadas
BinaryAccuracy	Accuracy adaptada a salida sigmoid binaria	Clasificación binaria
CategoricalAccuracy	Accuracy para etiquetas one-hot	Multiclase con categorical crossentropy
SparseCategorical-Accuracy	Accuracy para etiquetas enteras	Multiclase con sparse categorical crossentropy
TopK CategoricalAccuracy	Acierto si la clase real está entre las K más probables	Clasificación con muchas clases (ImageNet)
Precision	$TP / (TP + FP)$	Minimizar falsos positivos
Recall	$TP / (TP + FN)$	Minimizar falsos negativos
F1-score	Media armónica de precision y recall	Clases desbalanceadas
AUC	Área bajo la curva ROC	Clasificación binaria con probabilidades
MSE / MAE	Error cuadrático / absoluto medio	Regresión

Parte 2

Visión por Computadora: Transfer Learning y Beckhoff

10 Introducción

Esta guía constituye la **segunda parte** del laboratorio combinado de Inteligencia Artificial y Visión por Computadora, y aporta **10 puntos** a la nota total del laboratorio (sobre 20).

Partiendo de los fundamentos teóricos desarrollados en la Guía 1 (arquitectura de las CNN, proceso de entrenamiento, evaluación y preprocesamiento), en esta guía se introduce el *aprendizaje por transferencia* como solución a las limitaciones del entrenamiento desde cero y se construye un **sistema completo de control de calidad por visión por computadora**: una pareja de estudiantes capturará su propio dataset de tapitas de tres colores, entrenará un modelo basado en MobileNetV2 mediante aprendizaje por transferencia, e integrará el modelo entrenado con un computador industrial Beckhoff vía el protocolo ADS para disparar la acción de un brazo robótico colaborativo.

Objetivos del laboratorio:

- Construir y curar un dataset de imágenes industriales propio.
- Entrenar un modelo de clasificación mediante *transfer learning* con MobileNetV2.
- Aplicar *feature extraction* y *fine-tuning* como estrategias de adaptación al dominio.
- Comunicar un programa Python con un controlador Beckhoff mediante el protocolo ADS y la librería pyads.
- Integrar el clasificador con la lógica de control del IPC Beckhoff para producir un sistema de inspección automatizada.
- Comprender el rol de la inteligencia artificial y los robots colaborativos en el paradigma de Industria 5.0.

Conocimientos previos requeridos (de la Guía 1):

- Arquitectura de una red neuronal convolucional.
- Proceso de entrenamiento y evaluación de modelos.
- Preprocesamiento, aumentación de datos y regularización.

Materiales:

- Computadora con Python 3.10 o superior y entorno virtual del laboratorio activo.
- Cámara web USB o cámara IP (o cámara del teléfono móvil de los integrantes).
- Tapitas de plástico de tres colores: **rojo**, **azul** y **amarillo**.
- Computador industrial Beckhoff con TwinCAT 3.

- Brazo robótico colaborativo del laboratorio.
- Librerías tensorflow, opencv-python, pyads, numpy, matplotlib.

Recapitulación del pipeline. La Figura 13 reproduce el flujo end-to-end del sistema. La presente guía cubre las etapas *Modelo CNN*, *Predicción* y *3 BOOLs ADS*, y su integración con la salida digital del IPC Beckhoff que dispara el ciclo del brazo robótico.

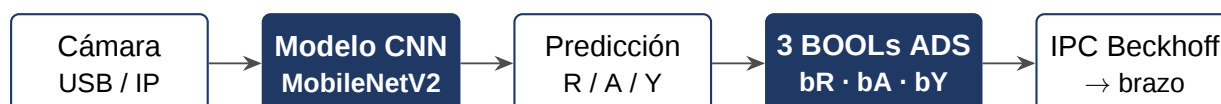


Figura 13. Pipeline end-to-end del laboratorio combinado. En la Guía 2 se construyen, integran y ponen en operación todos los bloques.

11 Industria 5.0 y robots colaborativos

11.1 Del paradigma 4.0 al 5.0

La automatización industrial ha atravesado distintas etapas de transformación. Desde la mecanización con la máquina de vapor (Industria 1.0), pasando por la electricidad y la producción en masa (2.0), la automatización con electrónica e informática (3.0), hasta la actual Industria 4.0 que integra de manera intensiva el Internet de las Cosas (IoT), la robótica avanzada, la computación en la nube y la inteligencia artificial.

La emergente **Industria 5.0** introduce una nueva dimensión: ya no se busca únicamente mejorar la productividad, sino integrar valores humanos, éticos y sostenibles en el entorno industrial. En lugar de reemplazar al operario, la Industria 5.0 propone una *colaboración estrecha entre humanos y máquinas*, donde cada uno aporta lo que mejor sabe hacer. La inteligencia artificial juega un papel central: no solo automatiza tareas, sino que se convierte en una herramienta que potencia las capacidades humanas mediante asistencia en la toma de decisiones, personalización de productos y anticipación de fallos.

11.2 Robots colaborativos (cobots)

Los **robots colaborativos** (o *cobots*) son la materialización tecnológica del paradigma 5.0. A diferencia de los robots industriales tradicionales — confinados a celdas cerradas por razones de seguridad —, los cobots están diseñados para trabajar junto a los operarios en el mismo entorno físico, compartiendo tareas, aprendiendo patrones de comportamiento y adaptándose a las decisiones humanas en tiempo real (Figura 14).

11.3 Normativa de seguridad: ISO/TS 15066

La integración de cobots en entornos donde interactúan con seres humanos exige el cumplimiento de normativas específicas. Aunque la Industria 5.0 aún no está formalmente normada por organismos internacionales, las tecnologías que la habilitan ya están sujetas a estándares establecidos. Las normas **ISO 10218-1** e **ISO 10218-2** establecen



Figura 14. Espacio de trabajo del laboratorio. Un brazo robótico colaborativo opera sobre una superficie con tapitas de colores que deben ser clasificadas y manipuladas. Fuente: elaboración propia.

los requisitos para el diseño y la integración segura de robots industriales. La especificación técnica **ISO/TS 15066** es particularmente relevante para aplicaciones colaborativas: define cuatro modos posibles de colaboración (parada supervisada por seguridad, guiado manual, monitoreo de velocidad y separación, y limitación de potencia y fuerza). Este último modo es el más utilizado en la práctica, e impone los límites cuantitativos de fuerza y presión presentados en la Tabla 1.

Cuadro 1. Límites de fuerza y presión según ISO/TS 15066 para contacto cuasi-estático con distintas partes del cuerpo humano.

Parte del cuerpo	Fuerza máxima (N)	Presión máxima (N/cm ²)
Dedo	140	100
Mano	190	160
Antebrazo	150	140
Cabeza (frontal)	65	130
Cabeza (lateral)	90	100
Muslo	210	150
Abdomen	110	140

12 Reutilización del dataset de tapitas

El dataset de tapitas se construyó en la **Guía 1 (Notebook 3)**: cada pareja grabó un video por color, extrajo los *frames* y los organizó en `data/tapitas/` siguiendo la convención de `flow_from_directory` (subcarpetas por clase). En esta guía se **reutiliza ese mismo dataset**, ahora con aprendizaje por transferencia, para contrastar el desempeño frente a la CNN entrenada desde cero del Notebook 3.

Además de los tres colores, el dataset incluye una cuarta clase fondo (imágenes sin tapita), de modo que el modelo tenga **cuatro salidas** y no se vea forzado a elegir un color cuando no hay objeto frente a la cámara. La clase fondo es puramente defensiva: **no activa ninguna salida del PLC** (solo los tres colores tienen una variable BOOL asociada).

Si el dataset resultó escaso o poco variado, esta es una buena oportunidad para ampliarlo: añadir muestras con mayor variación de iluminación, fondo y ángulo mejora la robustez del modelo. Se mantiene la división 80 % entrenamiento / 20 % validación, procurando que las muestras de validación sean *independientes* (de videos o sesiones de captura distintas), no frames consecutivos del mismo video.

13 Aprendizaje por transferencia

13.1 Motivación

Como se constató al cierre de la Guía 1, entrenar una CNN desde cero requiere típicamente conjuntos de datos muy grandes (decenas o cientos de miles de imágenes) y tiempo considerable de cómputo. Cuando el problema de interés cuenta con un dataset reducido —decenas o pocos cientos de imágenes por clase, como las tapitas capturadas por la pareja— una CNN entrenada desde cero tiende a sobreajustar y a generalizar pobremente, incluso aplicando aumentación de datos.

El **aprendizaje por transferencia** (*transfer learning*) explota la observación de que las primeras capas de una CNN aprenden detectores de bordes, texturas y patrones de bajo nivel que son *universales*: útiles no solo para la tarea original sobre la que la red fue entrenada, sino para casi cualquier tarea de visión por computadora. La estrategia consiste en partir de una CNN ya entrenada sobre un dataset masivo (típicamente *ImageNet*, con ≈ 1.3 millones de imágenes en 1000 clases) y *reutilizar* sus capas como extractor de características para nuestro problema particular (Figura 15).

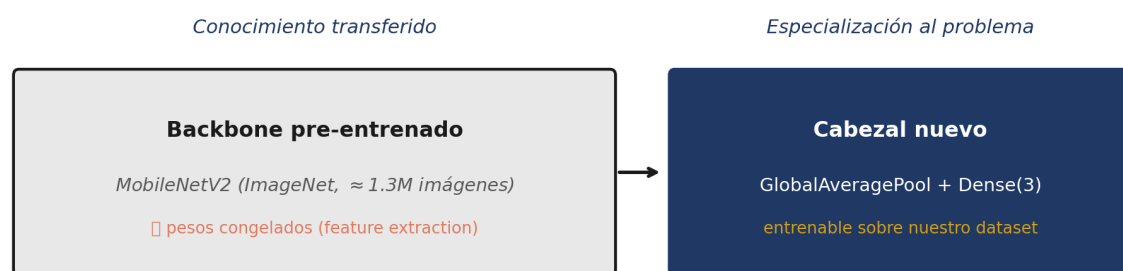


Figura 15. Esquema de aprendizaje por transferencia. El *backbone* pre-entrenado actúa como extractor universal de características; el cabezal nuevo se entrena para especializar la red al problema específico (aquí, la clasificación de tapitas por color). Fuente: elaboración propia.

13.2 MobileNetV2 como backbone

MobileNetV2 es una arquitectura propuesta por Sandler et al. (Google, 2018) diseñada específicamente para dispositivos con recursos limitados (móviles, sistemas embebidos, computadoras industriales). Sus características principales son:

- Aproximadamente 3.5 millones de parámetros (frente a ~ 60 millones de VGG-16 o ~ 25 millones de ResNet-50).
- Tamaño en disco ≈ 14 MB, ejecutable en CPU sin GPU.
- Bloques con conexiones residuales invertidas y convoluciones *depthwise separable*, que reducen drásticamente el cómputo respecto a CNN tradicionales.
- Entrada estándar de $224 \times 224 \times 3$ píxeles.
- Pesos pre-entrenados sobre ImageNet, disponibles directamente desde el módulo `keras.applications`.

Esta combinación de tamaño compacto y desempeño competitivo la convierte en el *backbone* preferido para sistemas de control de calidad embebidos, como el que se implementa en esta guía.

13.3 Feature extraction y fine-tuning

Existen dos modalidades principales de aprendizaje por transferencia, que en este laboratorio se aplican de forma secuencial:

1. Feature extraction (extracción de características). Se congelan todos los pesos del backbone pre-entrenado (no se actualizan durante el entrenamiento) y únicamente se entrena un nuevo cabezal de clasificación adaptado al problema específico. El backbone actúa como un extractor de características fijo. Es la modalidad más rápida y la recomendada como primer paso.

2. Fine-tuning (ajuste fino). Tras completar la fase anterior, se descongelan los últimos bloques del backbone (típicamente las capas más profundas, las que codifican características de alto nivel) y se continúa el entrenamiento con una tasa de aprendizaje *muy pequeña* (del orden de 10^{-5}). Esto permite que esas capas se especialicen ligeramente al dominio del problema sin destruir el conocimiento general adquirido.

Cuándo usar cada modalidad. En general:

- Dataset pequeño y muy similar al dominio original (e.g., otra tarea de objetos cotidianos): *feature extraction* suele ser suficiente.
- Dataset pequeño pero dominio distinto (imágenes médicas, satelitales): combinar *feature extraction* primero y luego *fine-tuning* de las últimas capas.
- Dataset grande ($\sim 10^4$ imágenes o más): *fine-tuning* completo puede ofrecer mejor desempeño.

14 Entrenamiento con Transfer Learning

En esta sección se desarrolla la implementación práctica del aprendizaje por transferencia presentado en la sección anterior (Sección 13). La estrategia se divide en dos fases: *feature extraction* con el backbone congelado, seguida de *fine-tuning* de las últimas capas.

14.1 Generadores con aumentación de datos

A diferencia de la Guía 1, los generadores ahora incorporan aumentación de datos. Es importante notar que para clasificación por color *no se deben aplicar transformaciones que alteren el matiz*; solo se aplican transformaciones geométricas y de iluminación moderadas (**Script 1**).

Script 1. Generadores con aumentación adecuada para clasificación por color.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

IMG_SIZE = 224
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.15,
    horizontal_flip=True,
    fill_mode='nearest'
)
val_datagen = ImageDataGenerator(rescale=1./255)

train_set = train_datagen.flow_from_directory(
    'data/tapitas/train',
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)
val_set = val_datagen.flow_from_directory(
    'data/tapitas/validation',
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)
```

14.2 Fase 1: Feature extraction

Se carga MobileNetV2 pre-entrenado sobre ImageNet sin su cabezal de clasificación, se congelan todos sus pesos, y se construye un cabezal nuevo con tres salidas (**Script 2**).

Script 2. Construcción del modelo de Transfer Learning con MobileNetV2.

```

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam

NUM_CLASSES = 4 # amarillo, azul, fondo, rojo

# 1. Cargar backbone pre-entrenado
backbone = MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights='imagenet'
)
backbone.trainable = False # congelar el backbone

# 2. Construir el cabezal nuevo
entrada = backbone.input
x = backbone.output
x = GlobalAveragePooling2D()(x)
x = Dropout(0.3)(x)
salida = Dense(NUM_CLASSES, activation='softmax')(x)

modelo = Model(inputs=entrada, outputs=salida)
modelo.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
modelo.summary()

```

El entrenamiento de esta fase utiliza una tasa de aprendizaje **estándar** (típicamente 10^{-3}), pues solo se está ajustando el cabezal — los pesos del backbone permanecen fijos (**Script 3**).

Script 3. Entrenamiento de la fase de feature extraction.

```

history_fe = modelo.fit(
    train_set,
    epochs=15,
    validation_data=val_set
)

```

14.3 Fase 2: Fine-tuning

Tras la fase de *feature extraction*, se descongelan los últimos bloques de MobileNetV2 (no toda la red) y se continúa el entrenamiento con una tasa de aprendizaje *muy pequeña* (**Script 4**). Esto permite que esas últimas capas se especialicen ligeramente al dominio del problema sin destruir el conocimiento general del backbone.

Script 4. Fine-tuning: descongelar los últimos bloques y reentrenar con LR pequeño.

```

# Descongelar el backbone completo y luego congelar todo
# excepto los ultimos 30 layers (los más profundos / específicos)

```

```
backbone.trainable = True
for layer in backbone.layers[:-30]:
    layer.trainable = False

# Recompilar con LR pequeño (CRUCIAL para no destruir los pesos)
modelo.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_ft = modelo.fit(
    train_set,
    epochs=10,
    validation_data=val_set
)

# Guardar el modelo entrenado (formato .h5)
modelo.save('modelo_tapitas.h5')
```

Por qué un LR tan pequeño en fine-tuning. Si se utilizara la tasa de aprendizaje original (10^{-3}) durante fine-tuning, los gradientes generados por la pérdida sobre el dataset pequeño de tapitas serían *demasiado grandes* para las capas pre-entrenadas. Esto destruiría rápidamente las representaciones útiles aprendidas sobre ImageNet. La regla práctica es usar entre 10^{-5} y 10^{-4} .

15 Protocolo ADS y comunicación con Beckhoff

15.1 El protocolo ADS

El **ADS** (*Automation Device Specification*) es un protocolo de comunicación desarrollado por Beckhoff para facilitar el intercambio de datos entre componentes de un sistema de automatización industrial, especialmente cuando se utiliza la plataforma *TwinCAT*. Su diseño permite una integración sencilla entre controladores PLC, sistemas SCADA, interfaces HMI y aplicaciones personalizadas escritas en lenguajes como C++, C# o Python (Figura 16).

Características principales del ADS.

- **Acceso por nombre simbólico:** las variables del PLC se acceden por su nombre dentro de una *Global Variable List* (GVL), sin necesidad de direcciones de memoria.
- **Comunicación sobre TCP/IP:** opera sobre IP sockets, permitiendo comunicación local y remota.
- **Independencia del lenguaje:** librerías oficiales y de terceros para C++, C#, Python (pyads), MATLAB, entre otros.
- **Baja latencia:** diseñado específicamente para entornos de automatización, ofreciendo comunicación adecuada para control en tiempo real.

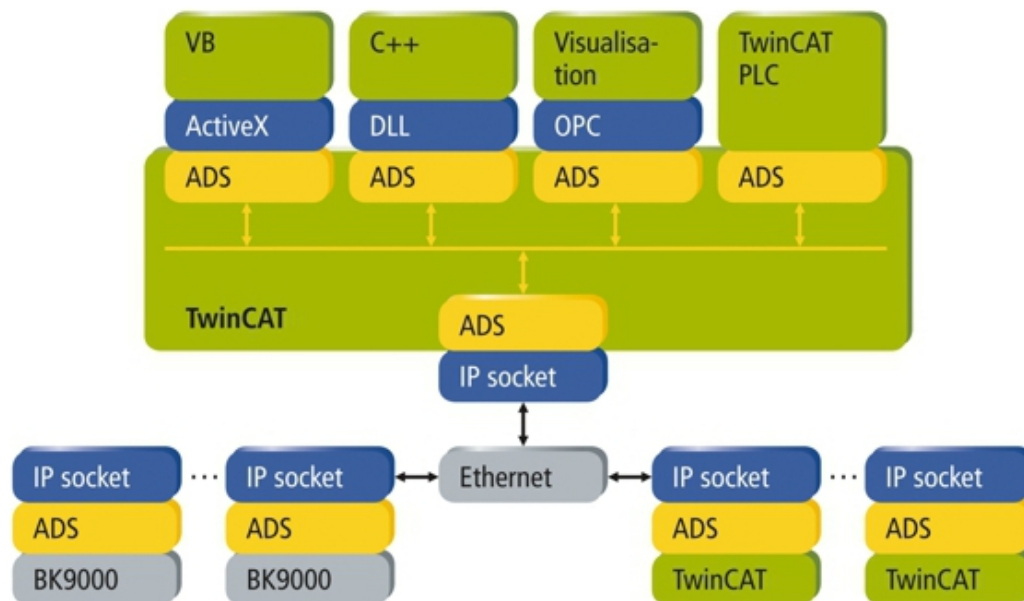


Figura 16. Arquitectura de comunicación del protocolo ADS. Múltiples aplicaciones cliente (en distintos lenguajes y nodos) se conectan al *router* ADS de TwinCAT mediante IP sockets sobre TCP/IP, accediendo a las variables del PLC por su nombre simbólico. Fuente: Beckhoff Automation (documentación oficial).

15.2 Configuración previa en TwinCAT

Para que el clasificador Python pueda escribir en las variables del IPC Beckhoff, deben cumplirse las siguientes condiciones del lado TwinCAT:

1. Declarar las variables en una GVL (Global Variable List) con nombre VARIABLES (mayúsculas).
2. Las variables deben ser de tipo BOOL: bRojo, bAzul, bAmarillo.
3. Habilitar el acceso ADS a estas variables (atributo {attribute 'TcLinkable'} o equivalente).
4. Configurar una *route* ADS entre la PC del estudiante y el IPC Beckhoff con su AMS NetID y puerto (851 por defecto).
5. Construir un **HMI** sencillo en TwinCAT (TwinCAT HMI o Visualization) con una **lámpara por color** —roja, azul y amarilla—, cada una enlazada a su variable BOOL (bRojo, bAzul, bAmarillo). Las lámparas permiten verificar visualmente, sin instrumentación adicional, que el clasificador escribe la variable correcta.

15.3 La librería pyads

Desde Python, la comunicación se establece a través de la librería pyads. El patrón básico se muestra en el **Script 5**.

Script 5. Conexión básica con un PLC Beckhoff mediante pyads.

```
import pyads
```

```
# Configuración del IPC (sustituir por el AMS NetID real del laboratorio)
AMS_NET_ID = '192.168.57.231.1.1'
PUERTO_ADS = 851

# Establecer conexión
ipc = pyads.Connection(AMS_NET_ID, PUERTO_ADS)
ipc.open()

# Escribir las tres variables booleanas
ipc.write_by_name('VARIABLES.bRojo', False, pyads.PLCTYPE_BOOL)
ipc.write_by_name('VARIABLES.bAzul', False, pyads.PLCTYPE_BOOL)
ipc.write_by_name('VARIABLES.bAmarillo', False, pyads.PLCTYPE_BOOL)

# Cerrar la conexión al terminar
ipc.close()
```

Importante. Los nombres utilizados en `write_by_name()` deben coincidir *exactamente* con los nombres simbólicos definidos en la GVL del proyecto de TwinCAT, respetando mayúsculas/minúsculas. Un error de nomenclatura produce `AdsError` en tiempo de ejecución.

16 Integración: del clasificador a la acción del brazo

16.1 Contrato de comunicación clasificador ↔ PLC

El sistema completo opera bajo el siguiente contrato:

1. El clasificador procesa el *frame* actual de la cámara.
2. Si la confianza de la predicción es superior a un umbral (típicamente 0.85), se escribe `True` en la variable `BOOL` correspondiente al color predicho, y `False` en las otras dos.
3. Si la confianza es menor al umbral, las tres variables se mantienen en `False` (no se dispara ninguna acción).
4. El programa PLC monitorea las tres variables. Al detectar un flanco ascendente en alguna de ellas, activa la salida digital física correspondiente.
5. La salida digital activa el ciclo del brazo robótico (movimiento de recolección y depósito en la zona correspondiente al color detectado).
6. Al finalizar el ciclo, el PLC reinicia las variables `BOOL`.

16.2 Bucle principal de inferencia y comunicación

El **Script 6** integra todos los componentes: carga del modelo entrenado, captura desde cámara, preprocesamiento, predicción y escritura de las tres variables `BOOL` en el IPC.

Script 6. Bucle principal: captura, predicción y escritura ADS.

```
import cv2
import numpy as np
```



```

import pyads
from tensorflow.keras.models import load_model
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

UMBRAL_CONFIANZA = 0.85
IMG_SIZE = 128 # mismo tamaño usado al entrenar (NB4)
CLASES = ['amarillo', 'azul', 'fondo', 'rojo'] # orden alfabético (igual que el entrenamiento)
VAR_POR_CLASE = { # 'fondo' no activa ninguna salida
    'rojo': 'VARIABLES.bRojo',
    'azul': 'VARIABLES.bAzul',
    'amarillo': 'VARIABLES.bAmarillo',
}

# Cargar modelo y abrir conexión ADS
modelo = load_model('modelo_tapitas.h5')
ipc = pyads.Connection('192.168.57.231.1.1', 851)
ipc.open()

cam = cv2.VideoCapture(0)
try:
    while True:
        ok, frame = cam.read()
        if not ok:
            break

        # Preprocesamiento MobileNetV2: BGR->RGB, redimensionar y normalizar
        # a [-1, 1] con preprocess_input (igual que en el NB4/NB6).
        img = cv2.cvtColor(cv2.resize(frame, (IMG_SIZE, IMG_SIZE)),
                           cv2.COLOR_BGR2RGB)
        x = np.expand_dims(preprocess_input(img.astype('float32')), axis=0)

        # Predicción
        pred = modelo.predict(x, verbose=0)[0]
        idx = int(np.argmax(pred))
        clase = CLASES[idx]
        confianza = float(pred[idx])

        # Decisión: se activa el BOOL del color detectado solo si la
        # confianza supera el umbral; el resto (y 'fondo') quedan en False.
        activo = VAR_POR_CLASE.get(clase) if confianza >= UMBRAL_CONFIANZA
        else None
        for var in VAR_POR_CLASE.values():
            ipc.write_by_name(var, var == activo, pyads.PLCTYPE_BOOL)

        # Visualización opcional
        etiqueta = f"{clase} ({confianza:.2f})"
        cv2.putText(frame, etiqueta, (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
        cv2.imshow('Clasificador', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
finally:
    cam.release()
    cv2.destroyAllWindows()
    # Limpiar variables antes de cerrar

```

```
for var in VAR_POR_CLASE.values():  
    ipc.write_by_name(var, False, pyads.PLCTYPE_BOOL)  
ipc.close()
```

17 Experiencia guiada

La experiencia integra todo el flujo del control de calidad inteligente: reutilización del dataset de tapitas (Guía 1), entrenamiento por transferencia, validación, comunicación con el IPC Beckhoff y demostración con el brazo. El trabajo práctico se apoya en los notebooks `parte2_vision/notebook4_transfer_learning.ipynb` (construcción y entrenamiento del modelo) y `parte2_vision/notebook5_probar_modelo.ipynb` (prueba con cámara en vivo) y `parte2_vision/notebook6_envio_beckhoff.ipynb` (envío de predicciones al IPC por pyads). Al igual que en la Guía 1, la pareja debe **ejecutar cada notebook de principio a fin, completar todas las celdas marcadas como «completar» y responder por escrito las preguntas conceptuales** que aparecen intercaladas. Cada notebook explica sus celdas en detalle; la presente sección describe qué se realiza en cada actividad y su peso en la evaluación. La rúbrica completa se encuentra en la *Ficha de Evaluación* del laboratorio: la Guía 2 aporta **10 puntos** que, sumados a los 10 de la Guía 1, conforman la nota del laboratorio sobre 20.

Actividad (herramienta)	Qué se realiza	Puntos
1. Feature extraction (notebook4_transfer_learning.ipynb, secc. 2–6)	Reutilizar el dataset de tapitas de la Guía 1, configurar los generadores con aumentación adecuada, construir el modelo con <i>backbone</i> MobileNetV2 congelado, entrenar el cabezal y evaluar con curvas y matriz de confusión.	3.0
2. Fine-tuning (notebook4_transfer_learning.ipynb, secc. 4–5)	Descongelar los últimos <i>layers</i> del <i>backbone</i> , recompilar con un <i>learning rate</i> pequeño y reentrenar; comparar el desempeño contra la fase anterior y contra la CNN del Notebook 3.	2.0
3. Conexión ADS y HMI (TwinCAT + pyads)	Declarar las tres variables BOOL en la GVL, construir un HMI con una lámpara por color y escribirlas desde Python (Script 5), verificando el encendido de la lámpara correspondiente.	2.0
4. Demostración integrada (notebook6_envio_beckhoff.ipynb + brazo)	Ejecutar el bucle de inferencia del NB6 (Script 6) con la cámara sobre el espacio de trabajo: cada color detectado activa su BOOL y dispara la acción correcta del brazo. Grabar un video de la demostración.	2.5
5. Discusión final	Reflexionar sobre cómo aplicar o adaptar el sistema en el proyecto integrador de automatización de línea de producción.	0.5
Total Guía 2		10.0

18 Anexos

Anexo B1. Variables del proyecto TwinCAT

Variable	Tipo	Descripción
bRojo	BOOL	Verdadero cuando se detecta una tapita roja con confianza mayor al umbral.
bAzul	BOOL	Verdadero cuando se detecta una tapita azul con confianza mayor al umbral.
bAmarillo	BOOL	Verdadero cuando se detecta una tapita amarilla con confianza mayor al umbral.

Estas variables deben declararse dentro de una GVL llamada VARIABLES y deben ser accesibles vía ADS. El nombre de la GVL forma parte del identificador simbólico

utilizado en `pyads.write_by_name()`.

Anexo B2. Tipos de datos pyads más utilizados

Constante pyads	Tipo TwinCAT correspondiente
PLCTYPE_BOOL	BOOL
PLCTYPE_INT	INT (16 bits con signo)
PLCTYPE_DINT	DINT (32 bits con signo)
PLCTYPE_REAL	REAL (32 bits coma flotante)
PLCTYPE_LREAL	LREAL (64 bits coma flotante)
PLCTYPE_STRING	STRING (cadena de texto)
PLCTYPE_TIME	TIME (en milisegundos)

Anexo B3. Diagnóstico de errores comunes

Error	Causa probable y solución
Missing ADS routes (7) target machine not found	La AMS <i>route</i> entre la PC y el IPC no está configurada (<code>open()</code> no la valida; el error aparece en la primera lectura/escritura). Crearla en <i>TwinCAT</i> (<i>bandeja</i>) → <i>Router</i> → <i>Edit Routes</i> → <i>Add Route</i> , o con <code>pyads.add_route_to_plc(...)</code> .
AdsError: symbol not found	El nombre de la variable en <code>write_by_name</code> no coincide con la GVL. Verificar mayúsculas/minúsculas y prefijo VARIABLES..
Predicciones siempre idénticas	El modelo está sobreajustado o el preprocesamiento no coincide entre entrenamiento e inferencia. Verificar la normalización.
Acción del brazo no se dispara	La variable se escribe correctamente pero el programa PLC no actúa sobre ella. Verificar la lógica del POU en TwinCAT.
Confianza siempre alta pero clase incorrecta	Dataset desbalanceado o de baja diversidad. Capturar más muestras en condiciones variadas.